

Лабораторная работа № 3. Измерение и тестирование пропускной способности сети. Воспроизводимый эксперимент

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Воспроизведение экспериментов посредством API Mininet	7
3.2	Построение графика по проведенному эксперименту	15
4	Выводы	19

Список иллюстраций

3.1	Создание подкаталога и копирование в него файла	7
3.2	Содержание скрипта lab_iperf3_topo.py	8
3.3	Запуск скрипта создания топологии	9
3.4	Просмотр элементов топологии	9
3.5	Внесение изменения в скрипт lab_iperf3_topo.py	10
3.6	Успешная отработка скрипта	10
3.7	Изменение скрипта для второго хоста	11
3.8	Успешная отработка скрипта с учетом второго хоста	11
3.9	Добавление записей об импорте классов	12
3.10	Изменение строки описания сети в скрипте, функции задания параметров для обоих хостов	13
3.11	Изменение функции параметров соединения	13
3.12	Запуск обоих скриптов	14
3.13	Создание копии скрипта lab_iperf3_topo2.py и помещение в подкаталог	15
3.14	Добавление записи в начало скрипта lab_iperf3.py	15
3.15	Внесение изменений в скрипт для хостов	16
3.16	Описание запуска на хостах и комментирование строк	16
3.17	Успешная отработка скрипта lab_iperf3.py	17
3.18	Построение графиков из JSON-файла и создание Makefile	17
3.19	Внесение изменений в Makefile	17
3.20	Успешная отработка Makefile	18

Список таблиц

1 Цель работы

Основной целью работы является знакомство с инструментом для измерения пропускной способности сети в режиме реального времени — iPerf3, а также получение навыков проведения воспроизводимого эксперимента по измерению пропускной способности моделируемой сети в среде Mininet.

2 Задание

1. Воспроизвести посредством API Mininet эксперименты по измерению пропускной способности с помощью iPerf3.
2. Построить графики по проведённому эксперименту.

3 Выполнение лабораторной работы

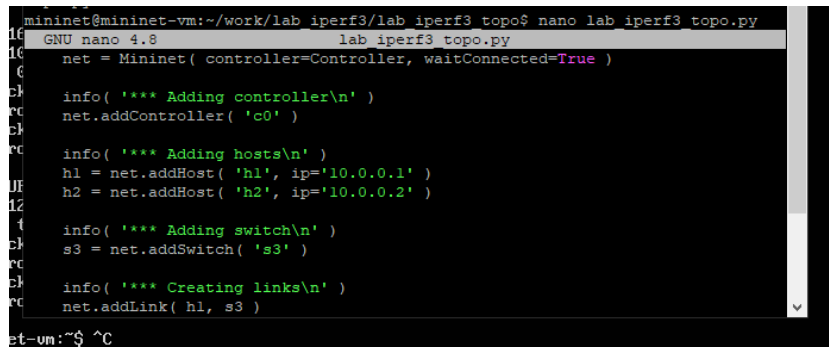
3.1 Воспроизведение экспериментов посредством API Mininet

1. С помощью API Mininet создайте простейшую топологию сети, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8:
- В каталоге /work/lab_iperf3 для работы над проектом создадим подкаталог lab_iperf3_topo и скопируйте в него файл с примером скрипта mininet/examples/emptynet.py, описывающего стандартную простую топологию сети mininet (рис. 3.1):

```
mininet@mininet-vm:~$ cd ~/work/lab_iperf3
mininet@mininet-vm:~/work/lab_iperf3$ mkdir lab_iperf3_topo
mininet@mininet-vm:~/work/lab_iperf3$ cd ~/work/lab_iperf3/lab_iperf3_topo
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cp ~/mininet/examples/emptynet.py ~/work/lab_iperf3/lab_iperf3_topo
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ mv emptynet.py lab_iperf3_topo.py
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$
```

Рис. 3.1: Создание подкаталога и копирование в него файла

- Изучим содержание скрипта lab_iperf3_topo.py (рис. 3.2):



```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ nano lab_iperf3_topo.py
GNU nano 4.8 lab_iperf3_topo.py
10 net = Mininet( controller=Controller, waitConnected=True )
11
12 info( '*** Adding controller\n' )
13 net.addController( 'c0' )
14
15 info( '*** Adding hosts\n' )
16 h1 = net.addHost( 'h1', ip='10.0.0.1' )
17 h2 = net.addHost( 'h2', ip='10.0.0.2' )
18
19 info( '*** Adding switch\n' )
20 s3 = net.addSwitch( 's3' )
21
22 info( '*** Creating links\n' )
23 net.addLink( h1, s3 )
24
mininet-vm:~$ ^C
```

Рис. 3.2: Содержание скрипта lab_iperf3_topo.py

Основные элементы:

- addSwitch(): добавляет коммутатор в топологию и возвращает имя коммутатора;
- addHost(): добавляет хост в топологию и возвращает имя хоста;
- addLink(): добавляет двунаправленную ссылку в топологию (и возвращает ключ ссылки; ссылки в Mininet являются двунаправленными, если не указано иное);
- Mininet: основной класс для создания и управления сетью;
- start(): запускает сеть;
- pingAll(): проверяет подключение, пытаясь заставить все узлы пинговать друг друга;
- stop(): останавливает сеть;
- net.hosts: все хосты в сети;
- dumpNodeConnections(): сбрасывает подключения к/от набора узлов;
- setLogLevel('info' | 'debug' | 'output'): устанавливает уровень вывода Mininet по умолчанию; рекомендуется info.

- Запустим скрипт создания топологии lab_iperf3_topo.py (рис. 3.3):

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
*** Running CLI
*** Starting CLI:
mininet>
```

Рис. 3.3: Запуск скрипта создания топологии

- После отработки скрипта посмотрим элементы топологии и завершим работу mininet (рис. 3.4):

```
mininet> net
h1 h1-eth0:s3-eth1
h2 h2-eth0:s3-eth2
s3 lo: s3-eth1:h1-eth0 s3-eth2:h2-eth0
c0
mininet> links
h1-eth0<->s3-eth1 (OK OK)
h2-eth0<->s3-eth2 (OK OK)
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=765>
<Host h2: h2-eth0:10.0.0.2 pid=768>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None pid=773>
<Controller c0: 127.0.0.1:6653 pid=758>
mininet> exit
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s3
*** Stopping 2 hosts
h1 h2
*** Done
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$
```

Рис. 3.4: Просмотр элементов топологии

- Внесем в скрипт lab_iperf3_topo.py изменение, позволяющее вывести на экран информацию о хосте h1, а именно имя хоста, его IP-адрес, MAC-адрес. Для этого после строки, задающей старт работы сети, добавим строку (рис. 3.5):

```
GNU nano 4.8 lab_iperf3_topo.py
from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( '*** Starting network\n' )
    net.start()
    print( "Host", h1.name, "has IP address", h1.IP(), "and MAC address", h1.MAC() )

    info( '*** Running CLI\n' )
    CLI( net )

^G Get Help ^C Write Out ^W Where Is ^R Cut Text ^J Justify ^C C
^X Exit ^O Read File ^N Replac ^M Page Top ^T To End
```

Рис. 3.5: Внесение изменения в скрипт lab_iperf3_topo.py

Здесь:

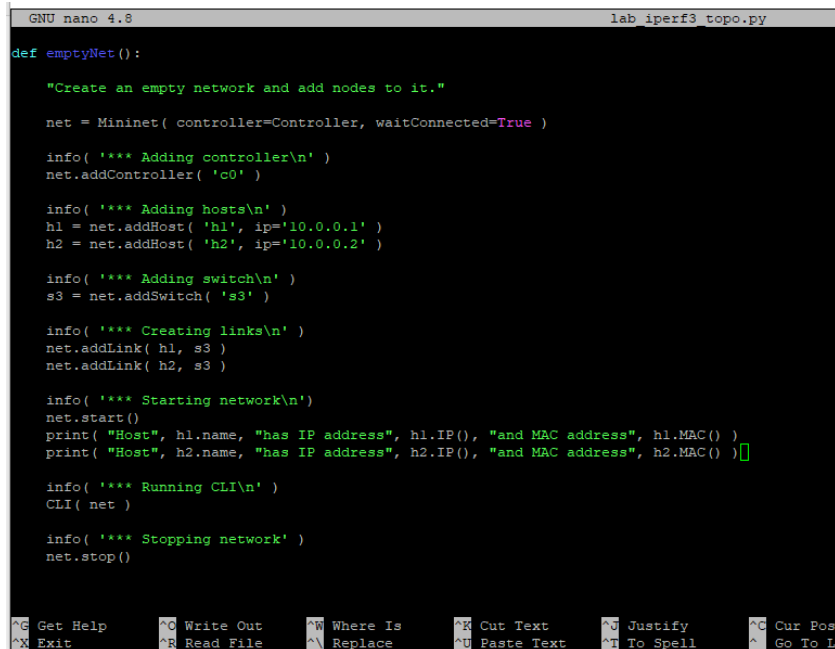
- IP() возвращает IP-адрес хоста или определенного интерфейса;
- MAC() возвращает MAC-адрес хоста или определенного интерфейса.

3. Проверим корректность отработки изменённого скрипта (рис. 3.6):

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ nano lab_iperf3_topo.py
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 6a:15:c9:20:1b:a8
*** Running CLI
*** Starting CLI:
mininet> 
```

Рис. 3.6: Успешная отработка скрипта

4. Изменим скрипт lab_iperf3_topo.py так, чтобы на экран выводилась информация об имени, IP-адресе и MAC-адресе обоих хостов сети (рис. 3.7). Проверим корректность отработки изменённого скрипта (рис. 3.8):



```
GNU nano 4.8 lab_iperf3_topo.py
def emptyNet():
    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

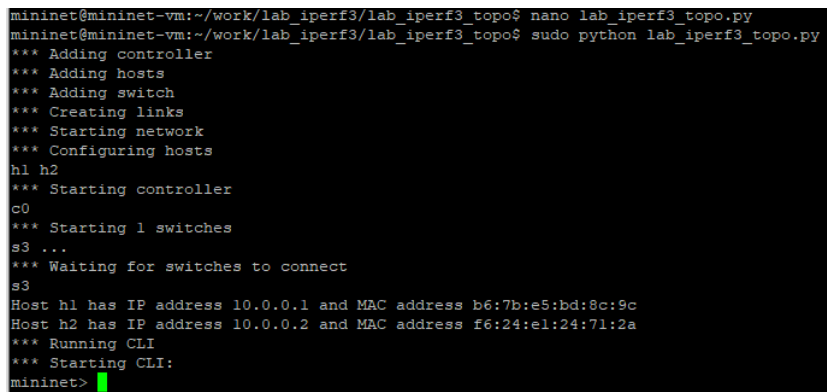
    info( '*** Creating links\n' )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( '*** Starting network\n' )
    net.start()
    print( "Host", h1.name, "has IP address", h1.IP(), "and MAC address", h1.MAC() )
    print( "Host", h2.name, "has IP address", h2.IP(), "and MAC address", h2.MAC() )

    info( '*** Running CLI\n' )
    CLI( net )

    info( '*** Stopping network\n' )
    net.stop()
```

Рис. 3.7: Изменение скрипта для второго хоста

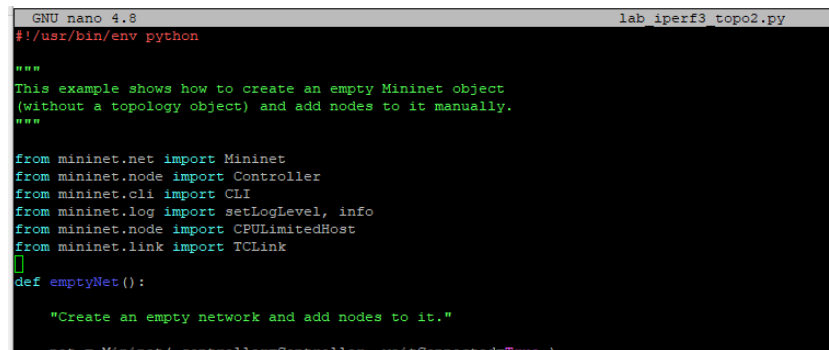


```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ nano lab_iperf3_topo.py
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address b6:7b:e5:bd:8c:9c
Host h2 has IP address 10.0.0.2 and MAC address f6:24:el:24:71:2a
*** Running CLI
*** Starting CLI:
mininet>
```

Рис. 3.8: Успешная отработка скрипта с учетом второго хоста

5. Mininet предоставляет функции ограничения производительности и изоляции с помощью классов CPULimitedHost и TCLink. Добавим в скрипт настройки параметров производительности.

- Сделаем копию скрипта `lab_iperf3_toro.py` с помощью команды: `cp lab_iperf3_toro.py lab_iperf3_toro2.py`.
- В начале скрипта `lab_iperf3_toro2.py` добавим записи об импорте классов `CPULimitedHost` и `TCLink` (рис. 3.9):



```

GNU nano 4.8                                lab_iperf3_toro2.py
#!/usr/bin/env python

"""
This example shows how to create an empty Mininet object
(without a topology object) and add nodes to it manually.
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.node import CPULimitedHost
from mininet.link import TCLink
[]
def emptyNet():
    "Create an empty network and add nodes to it."

    net = Mininet(controller=Controller, waitConnected=True)

```

Рис. 3.9: Добавление записей об импорте классов

- В скрипте `lab_iperf3_toro2.py` изменим строку описания сети, указав на использование ограничения производительности и изоляции.
- В скрипте `lab_iperf3_toro2.py` изменим функцию задания параметров виртуального хоста `h1`, указав, что ему будет выделено 50% от общих ресурсов процессора системы.
- Аналогичным образом для хоста `h2` зададим долю выделения ресурсов процессора в 45% (рис. 3.10):

```
GNU nano 4.8 lab_iperf3_topo2.py
#!/usr/bin/env python

"""
This example shows how to create an empty Mininet object
(without a topology object) and add nodes to it manually.
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.node import CPULimitedHost
from mininet.link import TCLink

def emptyNet():
    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True, host = CPULimitedHost, link = TCLink )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1', cpu=50 )
    h2 = net.addHost( 'h2', ip='10.0.0.2', cpu=45 )
```

Рис. 3.10: Изменение строки описания сети в скрипте, функции задания параметров для обоих хостов

- В скрипте lab_iperf3_topo2.py изменим функцию параметров соединения между хостом h1 и коммутатором s3 (рис. 3.11):

```
s3 = net.addSwitch( 's3' )

info( '*** Creating links\n' )
net.addLink( h1, s3, bw=10, delay='5ms', max_queue_size=1000, loss=10, use_htb=True )
net.addLink( h2, s3 )

info( '*** Starting network\n' )
```

Рис. 3.11: Изменение функции параметров соединения

Здесь добавляется двунаправленный канал с характеристиками пропускной способности, задержки и потерь:

- параметр пропускной способности (bw) выражается числом в Мбит;
- задержка (delay) выражается в виде строки с заданными единицами измерения (например, 5ms, 100us, 1s);
- потери (loss) выражаются в процентах (от 0 до 100);
- параметр максимального значения очереди (max_queue_size) выражается в пакетах;
- параметр use_htb указывает на использование ограничителя интенсивности входящего потока Hierarchical Token Bucket (HTB).

- Запустим на отработку сначала скрипт lab_iperf3_topo2.py, затем lab_iperf3_topo.py и сравним результат

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo2.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
(10.00Mbit 5ms delay 10.00000% loss) (10.00Mbit 5ms delay 10.00000% loss) *** Starting network
*** Configuring hosts
h1 (cfs 5000000/1000000us) h2 (cfs 4500000/1000000us)
*** Starting controller
c0
*** Starting 1 switches
s3 (10.00Mbit 5ms delay 10.00000% loss) ... (10.00Mbit 5ms delay 10.00000% loss)
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 1a:fb:9e:0e:dd:42
Host h2 has IP address 10.0.0.2 and MAC address 22:01:d4:ea:3f:50
*** Running CLI
*** Starting CLI:
mininet> exit
*** Stopping network*** Stopping 1 controllers
c0
(cfs -1/1000000us) (cfs -1/1000000us) *** Stopping 2 links
..
*** Stopping 1 switches
s3
*** Stopping 2 hosts
h1 h2
*** Done
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 42:5b:06:04:00:00
Host h2 has IP address 10.0.0.2 and MAC address 42:29:86:8a:57:30
*** Running CLI
*** Starting CLI:
mininet>
```

Рис. 3.12: Запуск обоих скриптов

При сравнении двух запусков видно, что скрипт lab_iperf3_topo2.py создает сеть с ограничениями пропускной способности (10.00Mbit), задержкой (5ms) и потерей пакетов (10.000000%), а также настраивает планировщик CFS для хостов h1 и h2 с конкретными квотами. Второй скрипт lab_iperf3_topo.py запускает сеть без этих ограничений и специальных настроек CFS, создавая базовую топологию.

3.2 Построение графика по проведенному эксперименту

6. Построим графики по проводимому эксперименту:

- Сделаем копию скрипта `lab_iperf3_topo2.py` и поместите его в подкаталог `iperf` (рис. 3.13):

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cp lab_iperf3_topo2.py lab_iperf3.py
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ mkdir -p ~/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ mv ~/work/lab_iperf3/lab_iperf3.py ~/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cd ~/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ ls -l
total 4
-rwxrwxr-x 1 mininet mininet 1347 Oct  7 07:52 lab_iperf3.py
```

Рис. 3.13: Создание копии скрипта `lab_iperf3_topo2.py` и помещение в подкаталог

- В начале скрипта `lab_iperf3.py` добавим запись (рис. 3.14):

```
GNU nano 4.8 lab_iperf3.py
"""
import time
from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.node import CPULimitedHost
from mininet.link import TCLink

def emptyNet():
    # Create an empty network and add nodes as is "
```

Рис. 3.14: Добавление записи в начало скрипта `lab_iperf3.py`

- Изменим код в скрипте `lab_iperf3.py` так, чтобы:
 - на хостах не было ограничения по использованию ресурсов процессора;
 - каналы между хостами и коммутатором были по 100 Мбит/с с задержкой 75 мс, без потерь, без использования ограничителей пропускной (рис. 3.15):

```

s3 = net.addSwitch( 's3' )

info( '*** Creating links\n' )
net.addLink( h1, s3, bw=100, delay='75ms' )
net.addLink( h2, s3, bw=100, delay='75ms' )

```

Рис. 3.15: Внесение изменений в скрипт для хостов

- После функции старта сети опишем запуск на хосте h2 сервера iPerf3, а на хосте h1 запуск с задержкой в 10 секунд клиента iPerf3 с экспортом результатов в JSON-файл, закомментируем строки, отвечающие за запуск CLI-интерфейса (рис. 3.16):

```

net.start()
print( "Host", h1.name, "has IP address", h1.IP(), "and MAC address", h1.MAC() )
print( "Host", h2.name, "has IP address", h2.IP(), "and MAC address", h2.MAC() )
info( '*** Starting network\n' )
info( '*** Traffic generation\n' )
h2.cmdPrint( 'iperf3 -s -D -l' )
time.sleep(10) # Wait 10 seconds for servers to start
h1.cmdPrint( 'iperf3 -c', h2.IP(), '-J > iperf_result.json' )
# info( '*** Running CLI\n' )
# CLI( net )

```

Рис. 3.16: Описание запуска на хостах и комментирование строк

Здесь после старта сети на хосте h2 в фоновом режиме запускается сервер iPerf3 с ключами -s (серверный режим), -D (демонизация) и -l (завершение после принятия одного соединения). Затем вводится задержка в 10 секунд для запуска сервера. На хосте h1 запускается клиент iPerf3, который подключается к IP-адресу h2, указанному через h2.IP(). Ключ -c задает клиентский режим, а -J делает вывод результатов в формате JSON, который перенаправляется в файл iperf_result.json. Строки, отвечающие за запуск интерактивного CLI-интерфейса Mininet, закомментированы, что позволяет выполнить тест автоматически без ручного вмешательства.

- Запустим на отработку скрипт lab_iperf3.py (рис. 3.17):


```

-rwxrwxr-x 1 mininet mininet 1347 Oct  7 07:52 lab_iperf3.py
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ nano lab_iperf3.py
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ sudo python lab_iperf3.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
(100.00Mbit 75ms delay) (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) *** Starting network
*** Configuring hosts
h1 (cfs -1/1000000us) h2 (cfs -1/1000000us)
*** Starting controller
c0
*** Starting 1 switches
s3 (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) ... (100.00Mbit 75ms delay) (100.00Mbit 75ms delay)
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 0a:c9:52:d3:82:d4
Host h2 has IP address 10.0.0.2 and MAC address 0a:ad:46:3f:d8:3c
*** Starting network
*** Traffic generation
*** h2 : ('iperf3 -s -D -1',)
*** h1 : ('iperf3 -c', '10.0.0.2', '-J > iperf_result.json')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s3
*** Stopping 2 hosts
h1 h2

```

Рис. 3.17: Успешная отработка скрипта lab_iperf3.py

- Построим графики из получившегося JSON-файла и создадим Makefile для проведения всего эксперимента (рис. 3.18):

```

mininet@mininet-vm:~/work/lab_iperf3/iperf3$ plot_iperf.sh iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ touch Makefile
mininet@mininet-vm:~/work/lab_iperf3/iperf3$

```

Рис. 3.18: Построение графиков из JSON-файла и создание Makefile

- В Makefile пропишем запуск скрипта эксперимента, построение графиков и очистку каталога от результатов (рис. 3.19):

```

GNU nano 4.8 Makefile
all: iperf_result.json plot

iperf_result.json:
    sudo python lab_iperf3.py

plot: iperf_result.json
    plot_iperf.sh iperf_result.json

clean:
    -rm -f *.json *.csv
    -rm -rf results

```

Рис. 3.19: Внесение изменений в Makefile

- Проверим корректность отработки Makefile (рис. 3.20):

```

mininet@mininet-vm:~/work/lab_iperf3/iperf3$ make
sudo python lab_iperf3.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
(100.00Mbit 75ms delay) (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) *** Starting network
*** Configuring hosts
h1 (cfs -1/1000000us) h2 (cfs -1/1000000us)
*** Starting controller
c0
*** Starting 1 switches
s3 (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) ... (100.00Mbit 75ms delay) (100.00Mbit 75ms delay)
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address fa:0f:df:9c:cf:bc
Host h2 has IP address 10.0.0.2 and MAC address 1a:93:20:81:35:33
*** Starting network
*** Traffic generation
*** h2 : ('iperf3 -s -D -1',)
*** h1 : ('iperf3 -c', '10.0.0.2', '-J > iperf_result.json')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s3
*** Stopping 2 hosts
h1 h2
*** Done
plot_iperf.sh iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$

```

Активация Windows
Чтобы активировать Windows, перейдите на [www.microsoft.com/windows/activation](#)

Рис. 3.20: Успешная отработка Makefile

4 Выводы

В результате выполнения данной лабораторной работы я познакомилась с инструментом для измерения пропускной способности сети в режиме реального времени — iPerf3, а также получила навыки проведения воспроизводимого эксперимента по измерению пропускной способности моделируемой сети в среде Mininet.